ESAT INNOVATION SCHOOL | ESCUELA SUPERIOR DE ARTE Y TECNOLOGÍA

ALGORITHMS AND DATA STRUCTURES
MEMORY

Nombre Asignatura: ALGORITMOS y ESTRUCTURAS DE DATOS
Modulo Asociado: 2PVG MA
Profesor: Aleix López
Curso: 2019/2020
Autor/es: Sergio Madaleno y Fede Sanjuan



Index

1. Hypotheses, time tables and conclusions based on the results obtained from the comparisons.

- **1.1 Inserts**
- **1.2 Extracts**
- **1.3 Concats**

2. Sorting Algorithms, what do they consist of? Times and analysis of the results.

- **2.1 Sorting Algorithms**
- **2.2 Our Results**

3. What advantages does the encapsulation and privatization of data have by porting the code to C++, as well as other relevant advantages of the port to C++.

4. Advantages and disadvantages of a moving head vector with twice the capacity as requested.

5. Differences of two search algorithms (depth and width)

6. Justify why ADTs are the basis of object-oriented programming and argue why you agree or disagree with this.

7. Logger explanation

8. Three advantages of why the ADT Memory Node allows us to obtain independence of the data with respect to the algorithms (separation of data vs algorithms).

1. Hypotheses, time tables and conclusions based on the results obtained from the comparisons.

1.1 Inserts

Insert	Num data	Elapsed Time	Average Time
Insert First			
Vector	10000	415882.000000 ms	41.588200 ms
List	10000	22152.099609 ms	2.215210 ms
DDList	10000	22741.699219 ms	2.274170 ms
Insert Last			
Vector	10000	7528.800293 ms	0.752880 ms
List	10000	21781.298828 ms	2.178130 ms
DDList	10000	24739.900391 ms	2.473990 ms
Insert At			
Vector	10000	526339.500000 ms	52.633950 ms
List	10000	149982.406250 ms	14.998241 ms
DDList	10000	61362.796875 ms	6.136280 ms

The provided data presents data into the performance characteristics of three different data structures vectors, lists, and double linked lists, across various insertion scenarios: inserting at the first position, inserting at the last position, and inserting at an arbitrary position. The analysis focuses on both the elapsed time, representing the total time taken for the insertions, and the average time per insertion for each data structure and insertion scenario.

When examining the impact of insertion position, a clear trend emerges: inserting at the beginning consistently results in the highest average and elapsed times across all data structures. This is likely due to the need for data shifting to accommodate the new element, a process that can become increasingly time consuming with larger datasets. Inserting at the last position consistently yields the lowest average and elapsed times, indicating that appending elements to the end of the data structure is more

efficient and involves fewer computational steps.

Comparing the performance of the three data structures, vectors generally exhibit slower insertion times compared to lists and doubly linked lists, especially for insertions at the beginning or at arbitrary positions. This is likely because vectors require contiguous memory allocation and may need to shift elements when inserting at positions other than the end. In contrast, both lists and doubly linked lists demonstrate more consistent performance across insertion positions, with doubly linked lists often showcasing slightly better efficiency. This superiority may stem from their ability to traverse both forward and backward, offering more efficient handling of insertions, particularly at the beginning or arbitrary positions.

Overall, the findings suggest that developers should carefully consider their specific requirements and usage patterns when selecting a data structure. For scenarios involving frequent insertions, particularly at the beginning or arbitrary positions, lists or doubly linked lists may offer better performance compared to vectors.

1.2 Extracts

Extract	Num data	Elapsed Time	Average Time
Extract First			
Vector	10000	7520.500000 ms	0.752050 ms
List	10000	8201.000000 ms	0.820100 ms
DDList	10000	8288.299805 ms	0.828830 ms
Extract Last			
Vector	10000	7516.799805 ms	0.751680 ms
List	10000	7343.399902 ms	0.734340 ms
DDList	10000	7480.500000 ms	0.748050 ms
Extract At			
Vector	10000	7658.700195 ms	0.765870 ms
List	10000	7544.700195 ms	0.754470 ms
DDList	10000	7712.700195 ms	0.771270 ms

The provided data shows on the efficiency of extraction operations from three different data structures: vectors, lists, and double linked lists. Extraction operations are analyzed across three scenarios: extracting from the first position, extracting from the last position, and extracting from an arbitrary position within the

data structure. The analysis focuses on both the elapsed time, representing the total time taken for extraction, and the average time per extraction for each data structure and extraction scenario.

The data suggests that the position from which elements are extracted has minimal impact on the efficiency of extraction operations across all three data structures. Whether extracting from the first, last, or an arbitrary position, the average and elapsed times remain relatively consistent. This finding contradicts expectations that extracting from certain positions, such as the beginning or end, might be faster or slower due to potential data shifting or traversal overhead.

Vectors show consistent performance across extraction positions. Lists and doubly linked lists also demonstrate stable performance across extraction scenarios, with doubly linked lists potentially offering a slight advantage due to their backward traversal capabilities.

These findings provide valuable information for developers when choosing a data structure for applications involving extraction operations. Regardless of the extraction position, vectors, lists, and doubly linked lists offer comparable efficiency. Therefore, developers can prioritize other factors, such as memory usage and traversal capabilities, when selecting the most suitable data structure for their specific application requirements. Overall, this analysis underscores the robustness and versatility of these data structures in handling extraction operations efficiently across various scenarios.

1.3 Concats

concat	Num data	Elapsed Time	Average Time
Concat			
list 1 & list 2	1000	747.000000 ms	0.747000 ms
list 1 & list 2	500	376.300018 ms	0.752600 ms
vector 1 & vector 2	1000	1188.400024 ms	1.188400 ms
vector 1 & vector 2	500	361.200012 ms	0.722400 ms
DLList 1 & DLList 2	1000	1749.099976 ms	1.749100 ms
DLList 1 & DLList 2	500	563.799988 ms	1.127600 ms

The provided data offers information of the efficiency of concatenating two data



structures: lists, vectors, and doubly linked lists and shows how the size of the data structures influences concatenation performance. The analysis focuses on both the elapsed time, representing the total time taken for concatenation, and the average time per concatenation operation for each data structure and dataset size.

Across all three data structures, concatenating smaller data structures consistently results in lower average and elapsed times compared to concatenating larger ones. This suggests that the efficiency of concatenation operations is closely tied to the size of the data structures involved.

For lists, the concatenation process demonstrates efficient performance, with both average and elapsed times remaining low, particularly for smaller datasets. Vectors also show efficient concatenation, although slightly higher average and elapsed times are observed compared to lists, especially for larger datasets. These findings underline the effectiveness of lists and vectors for concatenating data structures of varying sizes.

However, doubly linked lists present a different performance profile. While they maintain relatively efficient concatenation for smaller datasets, there is a notable increase in both average and elapsed times when concatenating larger datasets. This increase can be attributed to the traversal and linkage overhead inherent in doubly linked lists, which involve updating pointers and navigating through linked nodes during concatenation operations.

2. Sorting Algorithms, what do they consist of? Times and analysis of the results.

2.1 Sorting Algorithms

Sorting algorithms are important tools for arranging elements in a specific order, such as ascending or descending, and are fundamental for various applications, from organizing data for efficient searching to facilitating operations like binary search and merge operations. These algorithms consist of various techniques and approaches. Comparison based algorithms, including bubble sort, selection sort, insertion sort, etc., rely on comparing elements to determine their relative order.

Additionally, sorting algorithms have space complexity, representing the amount of additional memory required beyond the input dataset size. Overall, sorting algorithms form a diverse set of methods for organizing data efficiently, each with its own characteristics, advantages, and limitations. The choice of sorting algorithm depends on factors such as the size and nature of the dataset, desired time and space complexity, stability requirements, and adaptability to specific data patterns.

2.2 Our Results

Sorting Algorithm	Num data	Elapsed Time	Average Time
Bubble sort			
Vector	250	22090.298828 ms	22090.298828 ms
Vector	500	48438.898438 ms	48438.898438 ms
Vector	1000	122204.898438 ms	122204.898438 ms
Vector	1500	194090.500000 ms	194090.500000 ms
Selection Sort			
Vector	250	89587.000000 ms	89587.000000 ms
Vector	500	211812.187500 ms	211812.187500 ms
Vector	1000	289463.500000 ms	289463.500000 ms
Vector	1500	944951.000000 ms	944951.000000 ms

The provided data shows the performance of two sorting algorithms, Bubble Sort and Selection Sort, when applied to vectors of different sizes.

Bubble Sort:

For the Bubble Sort algorithm applied to vectors of size 250, 500, 1000, and 1500, the elapsed time increases as the size of the vector grows.
For example, with the vector size increasing from 250 to 500, the elapsed time approximately doubles. Similarly, the elapsed time more than doubles when moving from a vector size of 500 to 1000 and 1000 to 1500.
The average time per element also follows this trend, increasing linearly with the size of the vector.

Selection Sort:

Similarly, Selection Sort shows a significant increase in elapsed time as the size of the vector increases.
For example, with the vector size increasing from 250 to 500, the elapsed time approximately doubles. The trend continues as the vector size grows, with the elapsed time increasing substantially for larger datasets.
The average time per element also increases linearly with the size of the vector.

3. What advantages does the encapsulation and privatization of data have by porting the code to C++, as well as other relevant advantages of the port to C++.

Porting code to C++ brings several significant advantages, particularly in terms of encapsulation and data management. C++'s support for object oriented programming allows for the encapsulation of data within classes, shows better code organization and readability. By encapsulating data and operations within classes, developers can enforce data hiding by declaring data members as private. This prevents direct access to the data from outside the class, promoting data integrity and reducing the risk of unintended modifications. With encapsulation, developers can define well defined interfaces for interacting with the data, enhancing code modularity and maintainability.

C++ also offers support for inheritance and polymorphism, key features of OOP that facilitate code reuse and extensibility. By privatizing data within base classes and providing protected or public access to derived classes, developers can maintain encapsulation while allowing derived classes to inherit and manipulate data through defined interfaces. This hierarchical class structure promotes code reusability and scalability, as changes to the base class implementation can be propagated to derived classes without affecting their external behavior.

In addition to its OOP capabilities, C++ provides finer control over memory management compared to higher level languages. Developers can choose between manual memory allocation and deallocation using new and delete operators or utilize smart pointers for automatic memory management. This level of control over memory resources can lead to better performance and reduced memory overhead.



4. Advantages and disadvantages of a moving head vector with twice the capacity as requested.



Advantages:

- By doubling the capacity of the vector, the implementation reduces the frequency of resizing operations, which can be computationally expensive. This can lead to improved performance and reduced memory fragmentation over time, especially in scenarios with frequent insertions.

- Moving the head of the vector instead of shifting all elements when reaching capacity reduces the overhead associated with resizing operations. This approach minimizes the number of memory copies required, resulting in faster insertions and potentially lower memory usage.

- Keeping the head and tail pointers within a contiguous memory block enhances cache locality, as elements within the active range are stored closer together. This can lead to improved cache performance and reduced memory access latency, particularly in scenarios where data access patterns exhibit spatial locality.

Disadvantages:

- The implementation adds complexity to the vector class, introducing additional logic to handle the moving head and resizing operations. This complexity can increase code maintenance overhead and the likelihood of introducing bugs or logical errors.

- While doubling the capacity reduces the frequency of resizing operations, it may result in increased memory fragmentation over time. If the vector undergoes frequent insertions and deletions, unused memory space may accumulate between the active data range and the end of the vector, potentially leading to inefficient memory usage.

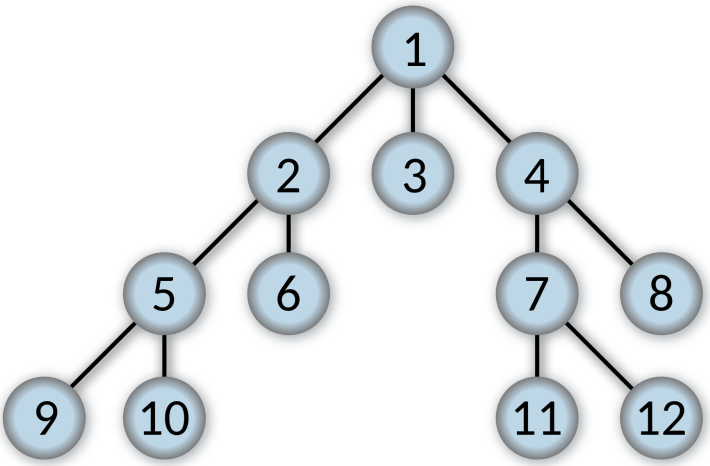
5. Differences of two search algorithms (depth and width)

Depth first search (DFS) and breadth first search (BFS) are two graph traversal algorithms, each with distinct characteristics and applications. DFS explores a graph by traversing as far down a branch as possible before backtracking. It starts at a chosen vertex and explores as far as possible along each branch before backtracking. This strategy resembles "exploring in depth," allowing DFS to reach the maximum depth of the graph before exploring other branches. DFS can be implemented using a stack data structure or recursion, efficiently managing backtracking and exploration. One notable advantage of DFS is its relatively low memory usage since it only needs to store information about the current path and the vertices on the current branch being explored.

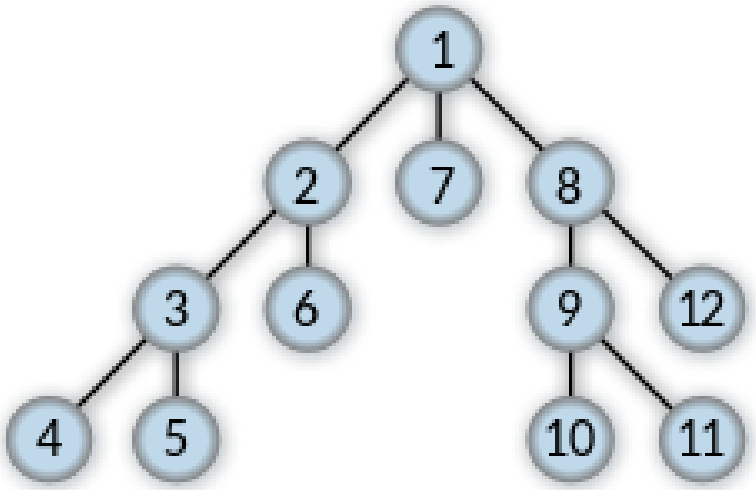
On the other hand, BFS explores a graph by visiting all vertices at the current depth level before moving to the next level. It starts at a chosen vertex and

explores all its neighbors before proceeding to the neighbors' neighbors, essentially "exploring in breadth." BFS typically uses a queue data structure to maintain the order of visited vertices, ensuring that vertices are visited in the order they were discovered. This property guarantees that BFS finds the shortest path to each vertex from the starting point, making it suitable for applications like finding the shortest path between two vertices, finding connected components, and solving puzzles. However, BFS tends to use more memory compared to DFS since it needs to store information about all vertices at the current depth level.

Breadth Search



Depth Search



Example:

Consider a maze solving scenario where you have to find a path from the entrance to the exit. DFS would systematically explore each possible path until it either reaches the exit or exhausts all possibilities. It is similar to a person traversing the maze by following one path until reaching a dead end and then backtracking to try another path. In contrast, BFS would explore all possible paths from the entrance level by level, ensuring that if there's a solution, it would find the shortest one. This is analogous to systematically exploring the maze level by level, marking the shortest path to the exit.

6. Justify why ADTs are the basis of object-oriented programming and argue why you agree or disagree with this.

Abstract Data Types serve as the foundation of object-oriented programming by providing a high level abstraction for organizing and manipulating data. ADTs encapsulate data and operations into a single unit, hiding implementation details and exposing only a well defined interface. This encapsulation promotes modularity, reusability, and abstraction, which are fundamental principles of OOP.

One argument supporting the idea that ADTs are the basis of OOP is that they align closely with the key concepts of OOP, such as encapsulation, inheritance, and polymorphism. Encapsulation, for example, is inherent in ADTs, as they encapsulate both data and operations within a single entity, shielding internal details from external access. This encapsulation fosters data integrity and code



maintainability, two essential aspects of OOP. Furthermore, ADTs facilitate inheritance by providing a blueprint for creating new data types based on existing ones, allowing for code reuse and promoting extensibility. Polymorphism, another OOP concept, is supported by ADTs through their ability to define common interfaces for different data types, enabling objects of different types to be treated interchangeably.

ADTs facilitate the design and implementation of complex systems by promoting modularization and separation of concerns. By encapsulating data and operations into distinct, self contained units, ADTs enable developers to focus on implementing specific functionalities without worrying about the intricacies of the underlying data structures. This approach enhances code organization, readability, and maintainability, making it easier to manage and extend large codebases.

However, some may argue against the assertion that ADTs are the basis of OOP, suggesting that OOP encompasses broader concepts beyond data abstraction. While ADTs provide a powerful mechanism for organizing data, OOP encompasses additional principles such as message passing, polymorphism, and dynamic binding, which may not be fully captured by traditional ADTs. Furthermore, some OOP languages prioritize message passing and dynamic dispatch over traditional ADTs, blurring the distinction between data and behavior.

7. Logger Explanation

The Logger structure in this code is a custom logging mechanism that uses a queue like storage to manage log entries.

Creating and Initializing the Logger
The LOGGER_create function is responsible for creating and initializing a new Logger instance. It takes a capacity parameter, which specifies the maximum number of log messages the logger can hold. It Creates the queue for storing log messages, and initializes the operations pointer with a set of predefined functions.

Writing to the Logger
The LOGGER_write function adds a new log message to the logger. It takes a Logger pointer and a string text as parameters. The function first checks if the logger is NULL. If not, it enqueues the text into the logger's storage. The function uses the enqueue method of the queue's operations to add the log message, ensuring the message is properly stored and managed.

Flushing and Destroying the Logger
The LOGGER_flush function removes and returns the oldest log message from the logger. It ensures the logger is not NULL before attempting to dequeue a message. This allows for processing of log messages in a First In First Out manner. The LOGGER_destroy function deallocates all resources associated with the logger. It calls the destroy method on the logger's storage to free the queue and then frees the logger itself.



Additional Logger Operations

- LOGGER_reset clears all log messages in the logger.
- LOGGER_resize changes the capacity of the logger's storage.
- LOGGER_capacity and LOGGER_length return the capacity and current number of messages in the logger, respectively.
- LOGGER_isEmpty and LOGGER_isFull check if the logger is empty or full.
- LOGGER_concat merges the contents of another logger into the current one.
- LOGGER_print outputs the current state of the logger for debugging purposes.

Each of these functions utilizes the corresponding methods from the queue's operations, ensuring that the logger's behavior is consistent and encapsulated within the Queue structure's functionality. This design allows the Logger to leverage the queue's management of data, while also providing a specialized interface for logging purposes.

8. Three advantages of why the ADT Memory Node allows us to obtain independence of the data with respect to the algorithms (separation of data vs algorithms).

The ADT Memory Node offers several advantages that enable independence of data from algorithms, facilitating the separation of concerns between data management and algorithmic operations:

- The Memory Node abstracts the underlying data storage mechanism, encapsulating data into a single unit with well defined operations. This encapsulation shields the internal representation of data from external algorithms, allowing algorithms to interact with data through a standardized interface. By encapsulating data within the Memory Node, algorithms become agnostic to the specific details of data storage, promoting modularity and reducing coupling between data structures and algorithms.

- The Memory Node enables algorithms to operate on data in a generic manner, independent of the underlying data structure. This flexibility allows algorithms to be reused across different data structures that implement the Memory Node interface. For example, algorithms designed to work with linked lists, arrays, or trees can be easily adapted to operate on Memory Nodes, as long as the required operations (e.g., insert, delete, retrieve) are supported. This promotes code reuse and simplifies maintenance, as algorithms can be developed and tested independently of specific data structures.

- The Memory Node abstracts common data operations, such as insertion, deletion, and retrieval, into a set of high level interface methods. These methods provide a standardized way for algorithms to interact with data,



hiding the implementation details of data manipulation algorithms. As a result, algorithms can focus on solving higher level problems without being burdened by low level data manipulation tasks. This abstraction of operations promotes separation of concerns by decoupling algorithmic logic from data manipulation logic, enhancing code clarity and maintainability.